

Informed Search: Greedy Best First Search and A* Search Part-2

Introduction

Imagine you're planning a road trip across a vast country, aiming to reach your destination as quickly and efficiently as possible. You wouldn't just pick the next road that looks like it's heading in the right direction; you'd consider how far you've already traveled and how much further you estimate you still have to go. This intuitive approach to problem-solving is precisely what informed search algorithms aim to replicate in artificial intelligence. Unlike uninformed search methods that explore blindly, informed search leverages additional knowledge, or "heuristics," to guide its exploration, making it significantly more efficient for complex problems.

This lecture delves into two pivotal informed search algorithms: Greedy Best-First Search and A Search. We will dissect their underlying principles, understand how they evaluate potential paths, and critically examine their strengths and weaknesses. By exploring a practical example on a map, we'll uncover why a seemingly intuitive approach like Greedy Best-First Search can sometimes lead to suboptimal outcomes or even infinite loops, paving the way for the more robust and widely used A algorithm. Understanding these concepts is crucial for anyone looking to design intelligent systems that can navigate complex environments effectively and efficiently.

Core Concepts

Term	Definition	Significance
$f(n)$	The evaluation function used by a search algorithm to determine the desirability of expanding a particular node n .	It dictates the order in which nodes are explored, fundamentally shaping the algorithm's behavior and performance.
$g(n)$	The actual cost incurred to reach node n from the initial starting state.	Represents the "past cost" or the cumulative cost of the path found so far to reach the current node.
$h(n)$		

Term	Definition	Significance
	An estimated cost (heuristic) from the current node n to the goal state.	Represents the "future cost" or an educated guess about how much more effort is needed to reach the goal from n .
Greedy Best-First Search	A search algorithm that prioritizes expanding the node n with the lowest estimated cost to the goal, using $f(n) = h(n)$.	It is "greedy" because it always picks the node that appears closest to the goal, without considering the cost already spent.
A* Search	A search algorithm that prioritizes expanding the node n with the lowest estimated total cost from the start to the goal, using $f(n) = g(n) + h(n)$.	It balances the cost incurred so far ($g(n)$) with the estimated future cost ($h(n)$), aiming for both efficiency and optimality.
Completeness	A property of a search algorithm indicating whether it is guaranteed to find a solution if one exists.	An algorithm that is not complete might fail to find a solution even when one is available, leading to infinite loops or premature termination.
Optimality	A property of a search algorithm indicating whether it is guaranteed to find the best (lowest cost) solution among all possible solutions.	An optimal algorithm ensures that the path found is the most efficient or cheapest one, which is often a critical requirement.
Heuristic Function	A function that estimates the cost from a given state to the goal state, providing an informed guess to guide the search.	The quality of the heuristic significantly impacts the efficiency of informed search algorithms; a good heuristic can dramatically reduce search time.

Term	Definition	Significance
Optimal Heuristic (h^*)	The true, actual cost from a given node n to the goal state.	Represents the perfect heuristic; if available, it would lead to the most efficient and optimal search, but it's usually unknown in practice.

Detailed Analysis

1. Greedy Best-First Search: The Allure of Proximity

Greedy Best-First Search (GBFS) is an informed search algorithm that operates on a simple, intuitive principle: always expand the node that appears to be closest to the goal. This "closeness" is determined solely by the heuristic function, $h(n)$, which estimates the cost from the current node n to the goal. Therefore, for GBFS, the evaluation function $f(n)$ is simply equal to $h(n)$. The algorithm's hope is that by always moving towards what seems like the goal, it will quickly reach the destination.

- **Intuition and Operation:**

- The core idea is to select the node for expansion that has the lowest $h(n)$ value.
- This means the algorithm prioritizes nodes that are heuristically closest to the goal, making a "greedy" choice at each step.
- For instance, if $h(n)$ represents the straight-line distance to the goal, GBFS would always move towards the city that is geographically closest to the destination.

- **Illustrative Example (Iasi to Oradea):**

- Consider a scenario where the goal is to travel from Iasi to Oradea.
- From Iasi, possible next cities are Vaslui and Neamt.
- If Neamt has a lower $h(n)$ (estimated distance to Oradea) than Vaslui, GBFS will choose Neamt.
- From Neamt, the only path might lead back to Iasi, which now appears to be the closest node to Oradea again (due to its $h(n)$ value).
- This creates a loop: Iasi $\rightarrow$ Neamt $\rightarrow$ Iasi $\rightarrow$ Neamt, and the algorithm gets stuck, never exploring other paths like Vaslui.

- **Properties and Limitations:**

- **Completeness:** GBFS is not complete for tree search because it can get stuck in infinite loops, as demonstrated by the Iasi-Neamt example. However, for graph search (with duplicate detection), it can be complete as it would eventually detect and avoid revisiting the same state in a loop.
- **Optimality:** GBFS is not optimal. Its greedy nature means it might find a path quickly, but there's no guarantee that this path will be the shortest or least costly. It prioritizes apparent closeness over actual cumulative cost.
- **Time and Space Complexity:** These are highly dependent on the quality of the heuristic function.
- If the heuristic is very good, GBFS can be very efficient.
- In the worst case, with a "crappy" or misleading heuristic, GBFS might explore almost all nodes, leading to a time complexity of $O(b^n)$ (where b is the branching factor and n is the depth of the solution), similar to uninformed search.
- Space complexity can also be $O(b^n)$ as the fringe (open list) might grow very large.
- **Myopic Nature:** The fundamental flaw of GBFS is its myopic focus on $h(n)$ alone. It completely ignores $g(n)$, the cost already incurred to reach the current node. This means it doesn't account for how "expensive" a path has become, only how "close" it seems to the goal.

2. A* Search: Balancing Past and Future Costs

The limitations of Greedy Best-First Search highlight a critical need: an algorithm that considers both the cost already paid and the estimated future cost. This is precisely what A Search achieves. A is designed to avoid expanding paths that are already too expensive, even if they appear close to the goal.

- **Addressing GBFS's Flaws:**
- The Iasi-Neamt loop in GBFS occurred because the algorithm only looked at $h(n)$. When returning to Iasi from Neamt, the $g(n)$ (cost from start) had increased significantly (e.g., $87 + 87 = 174$), but the $h(n)$ for Iasi remained the same, making it seem like a good choice again.
- Meanwhile, Vaslui, which might have had a slightly higher $h(n)$ initially but a lower $g(n)$ (e.g., 92), was ignored.
- The problem is that GBFS ignores the cumulative cost of the path, $g(n)$.
- **The A* Evaluation Function:**
- A* combines the "past cost" ($g(n)$) and the "future cost" ($h(n)$) into its evaluation function: $f(n) = g(n) + h(n)$.

- $g(n)$: The actual cost from the start node to node n .
- $h(n)$: The estimated cost from node n to the goal node.
- $f(n)$: The estimated total cost of the path from the start node to the goal node through node n .
- **Intuition of A*:**
 - A* aims to find the node n whose estimated total cost from the start to the goal, passing through n , is the lowest.
 - It's not just looking at how close n is to the goal (like GBFS), nor just how cheap it was to get to n (like Uniform Cost Search).
 - Instead, it seeks a balance, ensuring that paths that have become very long ($g(n)$ is high) are not preferred, even if their estimated remaining distance ($h(n)$) is low.
 - This prevents A* from getting stuck in loops or exploring unnecessarily long paths, as it continuously re-evaluates the total estimated cost.
- **Properties of A*:**
 - **Completeness:** A* is complete if the branching factor is finite and the cost of each step is greater than some small positive constant (i.e., not zero or negative costs).
 - **Optimality:** A is optimal if the heuristic function $h(n)$ is admissible (never overestimates the true cost to the goal) and consistent (satisfies the triangle inequality). These properties ensure that A finds the shortest path.
 - **Efficiency:** A* is generally much more efficient than GBFS, especially with a good heuristic. It explores fewer nodes than other optimal algorithms like Uniform Cost Search, making it "optimally efficient" among algorithms that use the same heuristic.
 - **Time and Space Complexity:** While often very efficient in practice, in the worst case, A* can still have $O(b^n)$ time and space complexity if the heuristic is poor or the search space is extremely large. However, a well-chosen heuristic can drastically reduce this.

A* search represents a significant advancement in informed search, providing a powerful and widely used algorithm that balances the desire for quick progress with the need for optimal solutions. Its ability to consider both past and future costs makes it a cornerstone of many AI applications, from pathfinding in video games to complex planning problems.

Key Takeaways

- [] $f(n)$ is the evaluation function, $g(n)$ is the cost from start to current node, and $h(n)$ is the estimated cost from current node to goal.
- [] Greedy Best-First Search uses $f(n) = h(n)$, prioritizing nodes that appear closest to the goal.
- [] Greedy Best-First Search is not complete for tree search and not optimal, as it can get stuck in loops and ignores the cost already incurred ($g(n)$).
- [] The time and space complexity of Greedy Best-First Search are highly dependent on the heuristic, potentially $O(b^n)$ in the worst case.
- [] A* Search uses $f(n) = g(n) + h(n)$, balancing the cost incurred so far with the estimated future cost.
- [] A* Search aims to find the node with the lowest estimated total cost from the start to the goal through that node.
- [] A* Search is complete and optimal under certain conditions (admissible and consistent heuristics, non-negative step costs).
- [] The quality of the heuristic function is crucial for the performance of both Greedy Best-First Search and A* Search.
- [] A* addresses the myopic nature of Greedy Best-First Search by incorporating $g(n)$, preventing it from getting stuck in local minima or loops due to high path costs.

Conclusion

The journey from Greedy Best-First Search to A* Search illustrates a fundamental principle in intelligent problem-solving: true efficiency and reliability often come from a balanced perspective. While the "greedy" approach of simply heading towards what looks closest might seem appealing for its speed, it often leads to pitfalls like infinite loops or suboptimal solutions, much like a traveler who always takes the shortest-looking road without considering traffic or tolls.

A Search, by contrast, introduces a crucial element of wisdom by combining the cost already expended ($g(n)$) with the estimated future cost ($h(n)$). This $f(n) = g(n) + h(n)$ evaluation function allows A to make more informed decisions, avoiding paths that have become too expensive even if they appear to lead directly to the goal. This balance makes A* a complete and optimal algorithm under reasonable conditions, making it a cornerstone for applications ranging from GPS navigation systems to robotics and game AI, where finding the best path efficiently is paramount. The lesson here is clear: effective problem-solving in AI, as in life, requires not just foresight, but also an acknowledgment of the journey already taken.